

Transaction

1. Feature Overview

Immutable ledger of credits/debits (including transfers). Package `com.bankone.transaction`: entity, repository, `TransactionService.record`, **CREDIT** on deposit and opening deposit, **DEBIT** on withdraw, **DEBIT+CREDIT** on transfer, `GET /accounts/{accountId}/transactions` (paged), and staff `GET /transactions` (global search UI).

Status: Implemented (write paths + account ledger UI + staff Transactions screen). Dashboard `todayTransactionCount` still stub.

2. Business Purpose

Provide audit-grade money movement history, feed reports/dashboard.

3. User Workflow

Implemented paths: Accounts list → Deposit / Withdraw / **Transfer**; open account / customer create with openingDeposit > 0 → CREDIT with narration “Opening deposit”; sidebar **Transactions**; Reports trends read the same ledger.

Portal transfers also post via `AccountService.transfer` when executed.

4. Execution Flow

List by account (implemented):

```
AccountController.getTransactions()  
    ↓  
TransactionServiceImpl.getByAccountId()  
    ↓  
AccountRepository.existsById()  
    ↓  
TransactionRepository.findByAccountAccountIdOrderByCreatedAtDesc()  
    ↓  
Page<TransactionResponse>
```

Deposit → ledger (implemented):

```

AccountController.deposit()
    ↓
AccountServiceImpl.deposit()
    ↓
AccountRepository.findById()
    ↓
assert ACTIVE + amount > 0
    ↓
increment availableBalance, ledgerBalance, creditCount, timestamps
    ↓
TransactionService.record(..., CREDIT, ...)
    ↓
TransactionRepository.save()
    ↓
AccountRepository.save()

```

5. Database Tables

`bank_transaction` (entity `Transaction`). FK `account_id` → `account`.

6. REST APIs

Method	Path	Roles
GET	<code>/accounts/{accountId}/transactions?page&size...</code>	ADMIN, EMPLOYEE, MANAGER

Hosted on `AccountController` (no separate transaction controller). Money-movement POSTs stay on Account API and write the ledger internally (`deposit` / `withdraw` / `transfer` / opening deposit).

7. Controllers

List endpoint on `AccountController`; package `com.bankone.transaction` has no dedicated controller yet.

8. Services

- `TransactionService` / `TransactionServiceImpl.record(...)` — validates account, type, amount > 0, balanceAfter; sets currency from account; persists via repository
- `getByAccountId(accountId, pageable)` — ensures account exists; returns paged `TransactionResponse`

9. Repositories

- `TransactionRepository` extends `JpaRepository<Transaction, Long>`
 - `findByAccountAccountIdOrderByCreatedAtDesc(Long)`
 - `findByAccountAccountIdOrderByCreatedAtDesc(Long, Pageable)`

10. DTOs

`TransactionResponse` for list API. Record path is still internal; deposit still uses Account DTOs.

11. Entities

- `Transaction` → table `bank_transaction`
 - `transactionId`, `account`, `transactionType`, `amount`, `balanceAfter`, `currencyCode`, `narration`, `createdAt` (**UTC Instant**, JSON with `Z`), `createdBy`

12. Utility Classes

None

13. Configuration Classes

None dedicated; deposit and list use existing `/accounts/**` security matchers.

14. Validation Rules

In `record`: account required; type required; amount > 0; balanceAfter required. Deposit path still enforces ACTIVE + amount > 0 before calling `record`.

Planned for debit: sufficient balance; currency match.

15. Security Rules

Write today via Account deposit and openAccount opening deposit (ADMIN/EMPLOYEE). List read aligns with accounts (ADMIN, EMPLOYEE, MANAGER).

16. Exception Handling

`IllegalArgumentException` from `record` / deposit; reuse `GlobalExceptionHandler`

17. Logging

None dedicated yet; planned structured log per posting

18. Audit Events

Ledger row **is** the financial audit (`created_by`, `created_at`, `balance_after`). Optional link to `audit` module for richer who/when metadata later.

19. Testing Strategy

- Deposit creates +1 CREDIT with matching amount and `balance_after`
- Withdraw insufficient funds → reject
- Transfer same-account / currency mismatch → reject
- Concurrent balance safety (pessimistic lock on withdraw/transfer)

20. Future Extension Guide

Next: beneficiary-linked transfers; standalone Transactions nav; wire dashboard

`todayTransactionCount`; richer statement PDF.

Future Modification Guide

Requirement: Expose transaction list API — done

Item	Detail
Files	<code>AccountController.java</code> , <code>TransactionService / Impl</code> , <code>TransactionResponse.java</code> , account-detail UI
Methods	<code>getTransactions</code> , <code>getByAccountId</code>
Impact	Paged ledger on account detail

Requirement: Write CREDIT on openAccount — done

Item	Detail
Files	<code>AccountServiceImpl.java</code>
Methods	<code>openAccount()</code> — after save call <code>transactionService.record(..., CREDIT, ...)</code>
Impact	Opening balances appear in ledger

Requirement: Add withdraw (DEBIT) — done

Item	Detail
Files	<code>AccountController</code> , <code>AccountServiceImpl</code> , withdraw dialog FE
Methods	<code>withdraw()</code> — lock, balance check, <code>record(..., DEBIT, ...)</code>
Impact	API + Accounts UI; Kafka email

Call hierarchy (deposit → record) — implemented

```
AccountController.deposit()
    ↓
AccountServiceImpl.deposit()
    ↓
AccountRepository.findById()
    ↓
TransactionService.record(..., TransactionType.CREDIT, ...)
    ↓
TransactionServiceImpl.record()
    ↓
TransactionRepository.save()
    ↓
AccountRepository.save()
```